



Week3_예습과제_고은비

Chapter4. 분류

1 분류(Classification)의 개요

2 결정 트리

결정 트리 모델의 특징

결정 트리 파라미터

결정 트리 모델의 시각화

결정 트리 과적합

3 앙상블 학습

보팅 유형 - Hard Voting과 Soft Voting

보팅 분류기 (Voting Classifier)

4 랜덤 포레스트

개요 및 실습

하이퍼 파라미터 및 튜닝

Chapter4. 분류

1 분류(Classification)의 개요



분류 (Classification) 란?

지도학습 : 레이블이 있는 데이터가 주어진 상태에서 학습하는 방식

>> 분류는 지도학습의 대표적 유형

1. 주어진 데이터의 피쳐, 레이블 값을 알고리즘으로 학습해 모델 생성
2. 새로운 데이터 값에 대한 미지의 레이블 값을 예측

>> 즉, 기존 데이터가 어떤 레이블에 속하는지 패턴을 알고리즘으로 인지한 뒤, 새롭게 관측된 데이터에 대한 레이블을 판별하는 것

분류 알고리즘 종류

- 베이즈(Bayes) 통계와 생성 모델에 기반한 나이브 베이즈(Naive Bayes)
- 독립변수와 종속변수의 선형 관계성에 기반한 로지스틱 회귀 (Logistic Regression)
- 데이터 균일도에 따른 규칙 기반의 결정 트리 (Decision Tree)
- 개별 클래스 간의 최대 분류 마진을 효과적으로 찾아주는 서포트 벡터 머신 (Support Vector Machine)
- 근접 거리를 기준으로 하는 최소 근접(Nearest Neighbor) 알고리즘
- 심층 연결 기반의 신경망 (Neural Network)
- 서로 다른(또는 같은) 머신러닝 알고리즘을 결합한 **앙상블(Ensemble)**



이번 장에서 집중적으로 다루는 방법

앙상블 기법 (Ensemble Method)

: 서로 다른/같은 알고리즘을 결합

분류에서 가장 각광 받는 방법 중 하나 (정형 데이터의 예측 분석)

- **배깅 (Bagging)**

- 랜덤 포레스트 (RandomForest)

: 배깅 방식의 대표적인 방식

>> 뛰어난 예측 성능, 빠른 수행 시간, 유연성 등으로 많은 전문가가 애용

- **부스팅 (Boosting)**

근래의 앙상블 기법은 부스팅 방식으로 발전 중

- 그래디언트 부스팅 (Gradient Boosting)

>> 뛰어난 예측 성능을 가지고 있지만, 수행 시간이 오래 걸림

- XgBoost (eXtra Gradient Boost)

- LightGBM

>> 기존 그래디언트 부스팅을 업그레이드 시킨 최신 기법들

- **스태킹 (Stacking) 기법**

: 앙상블의 앙상블

2 결정 트리



결정 트리?

앙상블의 기본 알고리즘으로 일반적으로 사용되는 알고리즘

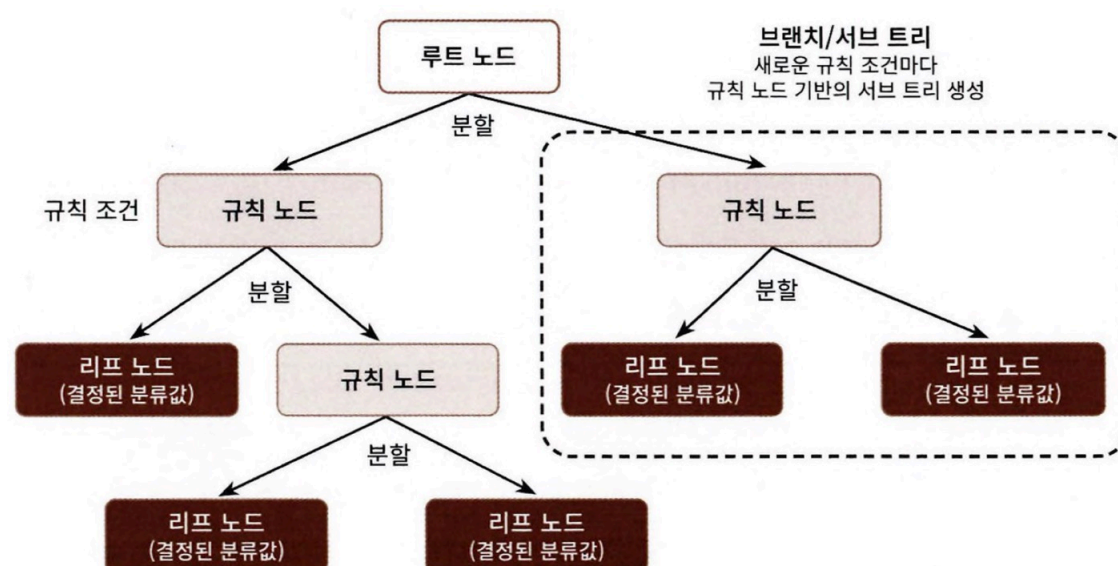
앙상블은 매우 많은 여러개의 약한 학습기 (예측 성능이 떨어지는 학습 알고리즘)를 결합해 확률적 보완과 오류가 발생한 부분에 대한 가중치를 계속 업데이트하며 예측 성능을 향상

▶ 결정트리가 좋은 약한 학습기가 됨

결정 트리의 구조

데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 트리(Tree) 기반의 분류 규칙을 만드는 것

▶ 어떤 기준을 바탕으로 규칙을 만들어야 가장 효율적인 분류가 될 것인가가 알고리즘의 성능을 크게 좌우



- 규칙 노드 (Decision Node) : 규칙 조건, 데이터 세트의 피쳐가 결합되어 만들어짐

- 리프 노드 (Leaf Node) : 결정된 클래스 값

- 서브 트리 (Sub Tree) : 새로운 규칙 조건마다 생성

▶ 많은 규칙, 트리의 깊이 (depth)가 깊어질수록 과적합으로 이어지기 쉬움

- ➡ 가능한 적은 결정노드로 최대한 많은 데이터 세트를 분류하도록 트리를 분할(Split) 해야 함
- ➡ 최대한 **균일한** 데이터 세트를 구성할 수 있도록 분할

균일한 데이터 세트란?



- 데이터 세트의 균일도는 데이터를 구분하는 데 필요한 정보의 양에 영향을 미침
 - C의 경우 하나의 데이터를 뽑았을 때, '검은 공'이라고 쉽게 예측 가능
 - A의 경우 혼잡도가 높고 균일도가 낮아 데이터를 판단하는데 있어 추가 정보가 필요
- 결정 노드는 **정보 균일도가 높은 데이터 세트를 먼저 선택**할 수 있도록 규칙 조건을 만듦
- **균일도 측정 방법**
 - 정보 이득(Information Gain)지수

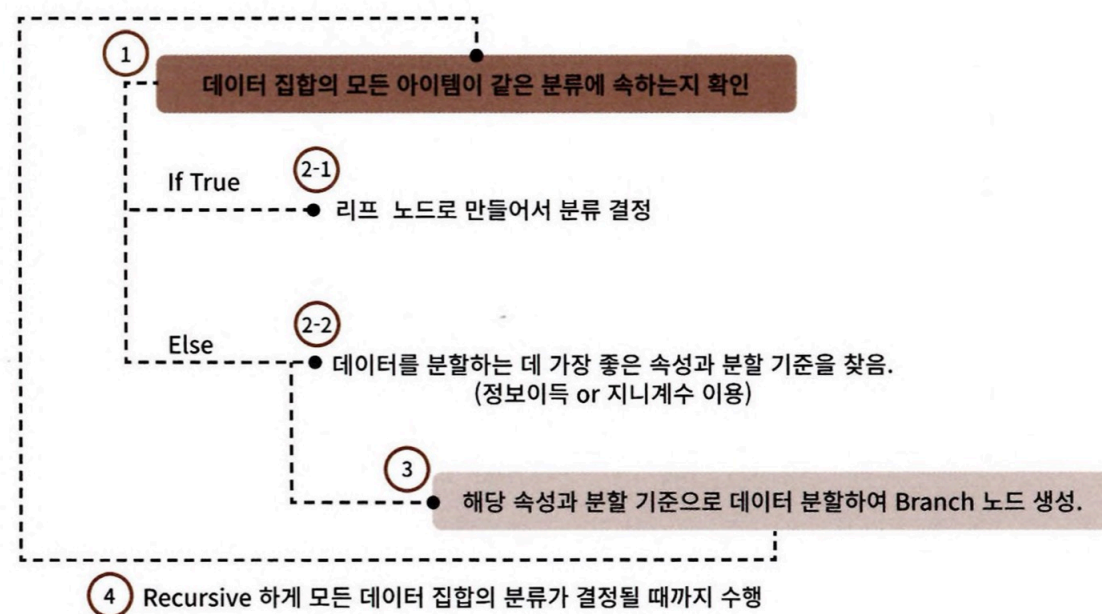
$$InformationGain = 1 - Entropy$$

1에서 엔트로피 지수를 뺀 값 (엔트로피는 주어진 데이터 집합의 혼잡도를 의미)
 ⇒ 정보 이득이 높은 속성을 기준으로 분할

◦ 지니 계수

경제학에서 불평등 지수를 나타낼 때 사용되는 계수 - 0이 가장 평등, 1로 갈수록 불평등
 ⇒ 지니 계수가 낮은 속성을 기준으로 분할

DecisionTreeClassifier - 사이킷런에서 구현된 결정 트리 알고리즘이 사용함



결정 트리 모델의 특징

장점

- 쉽다/직관적이다.
 - '균일도'라는 룰을 기반으로 결정

- 시각화로 표현 가능
- 피처의 사전 가공 영향도가 크지 않음
 - 정보의 균일도만 신경쓰면 됨

단점

- 과적합 가능성
 - 피처가 많고 균일도가 다양하게 존재할수록 트리의 깊이가 커지고 복잡해질 수 있음
 - 완벽한 규칙은 만들지 못하는 경우가 많음에도, 모델의 정확도를 높이기 위해 조건을 계속해 추가 → 복잡한 학습 모델이 됨
- ⇒ 트리의 크기를 사전에 제한하는 것이 오히려 성능 튜닝에 더 도움이 됨

결정 트리 파라미터

사이킷런은 결정트리 알고리즘을 구현한 `DecisionTreeClassifier` (분류를 위한 클래스)와 `DecisionTreeRegressor` (회귀를 위한 클래스) 제공

- CART (Classification And Regression Trees)알고리즘을 기반으로 함
- 두 클래스 모두 동일한 파라미터를 사용

파라미터 명	설명
<code>min_samples_split</code>	▪ 노드를 분할하기 위한 최소한의 샘플 데이터 수 ▪ 디폴트는 2 → 작게 설정할수록 분할되는 노드가 많아져 과적합 가능성 증가
<code>min_samples_leaf</code>	▪ 분할이 될 경우, 왼/오른쪽의 브랜치 노드에서 가져야할 최소한의 데이터 수 ▪ 값이 커질수록 노드 분할을 상대적으로 덜 수행 ▪ <code>min_samples_split</code>와 유사하게 과적합을 제어, 그러나 비대칭적 데이터의 경우 특정 클래스의 데이터가 극도로 작아질 수 있으므로 이런 경우 작게 설정해야함
<code>max_features</code>	▪ 최적의 분할을 위해 고려할 최대 피쳐 개수 (디폴트 = None) ▪ int 형 → 개수 / float형 → 퍼센트 ▪ sqrt : $\sqrt{\text{전체 피쳐 개수}}$ 만큼 선정 ▪ auto : sqrt와 동일 ▪ log : 전체 피쳐 중 $\log_2(\text{전체 피쳐 개수})$ 선정 ▪ None : 전체 피쳐 선정
<code>max_depth</code>	▪ 트리의 최대 깊이 규정 ▪ 디폴트 = None → 완벽하게 결정 값이 될 때까지 계속 깊이 증가 (<code>min_samples_split</code>보다 개수가 작아지면 중단) ▪ 깊이가 깊어지면 과적합할 수 있으므로 적절하게 제어 필요
<code>max_leaf_nodes</code>	▪ 말단 노드 (Leaf)의 최대 개수

결정 트리 모델의 시각화

→ **Graphviz** 패키지 사용

Graphviz

Please join the Graphviz forum to ask questions and discuss Graphviz. What is Graphviz? Graphviz is open source graph visualization software. Graph visualization is a way of representing structural information as diagrams of abstract graphs and networks. It has important applications in networking, bioinformatics, software engineering, database and web design, machine learning, and in visual interfaces for other technical domains.

<https://www.graphviz.org/>

- 그래프 기반의 dot 파일로 기술된 다양한 이미지를 쉽게 시각화 가능
- 사이킷런의 `export_graphviz()` API를 통해 인터페이스 가능
- 함수 인자로 학습을 끝낸 Estimator, 피쳐 이름 리스트, 레이블 이름 리스트 입력
 - 학습된 결정 트리 규칙을 트리 형태로 시각화


```

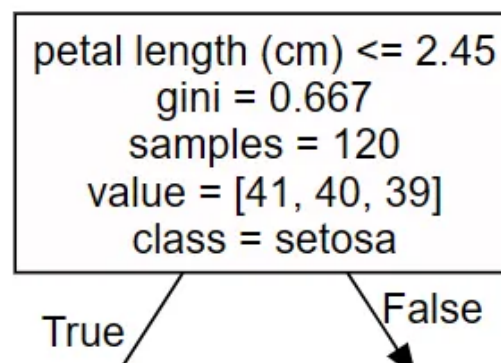
from sklearn.tree import export_graphviz
import graphviz

# export_graphviz()의 호출 결과로 out_file로 지정된 tree.dot 파일을 생성
export_graphviz(dt_clf, out_file="tree.dot", class_names=iris_data.target_names,
                feature_names = iris_data.feature_names, impurity=True, filled=True)

# 위에서 생성된 tree.dot 파일을 Graphviz가 읽어서 주피터 노트북상에서 시각화
with open("tree.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)

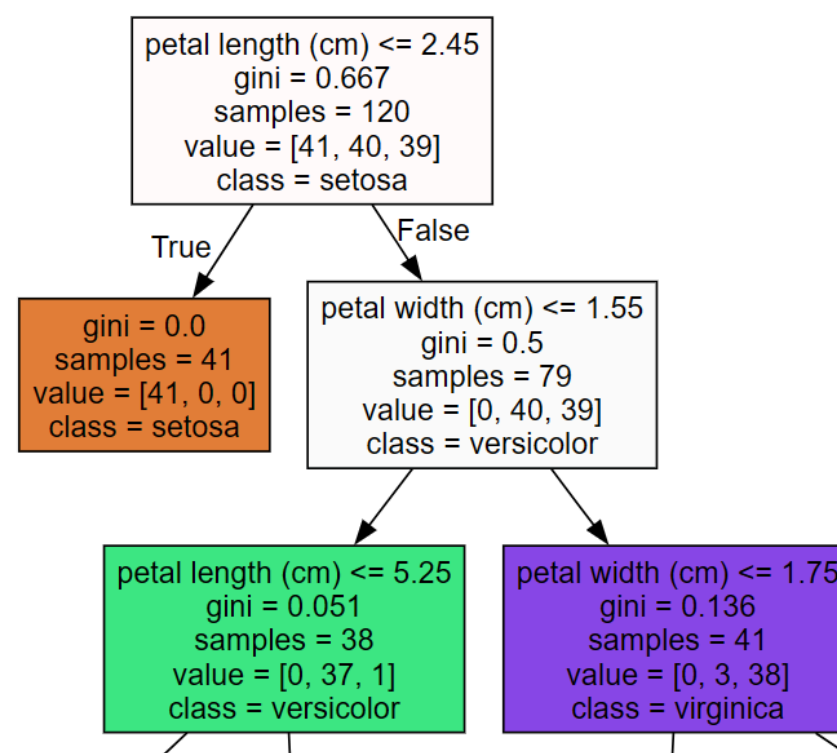
```

노드 내 지표의 의미



지표	의미
<code>petal length (cm) <= 2.45</code>	자식 노드를 만들기 위한 규칙 조건 → 조건이 없으면 리프 노드
<code>gini</code>	다음의 value로 주어진 데이터 분포에서의 지니계수
<code>samples</code>	현 규칙의 대상이 되는 데이터 건수
<code>value = []</code>	클래스 값 기반의 데이터 건수 e.g. 붓꽃 데이터 셋은 0:Setosa, 1:Versicolor, 2:Virginica 품종을 가리킴 → Value=[41,40,39] : 각 품종이 41, 40, 39개로 데이터가 구성되어 있다
<code>class = setosa</code>	하위 노드를 가질 경우, setosa가 41개로 제일 많다는 뜻

트리 구성 의미

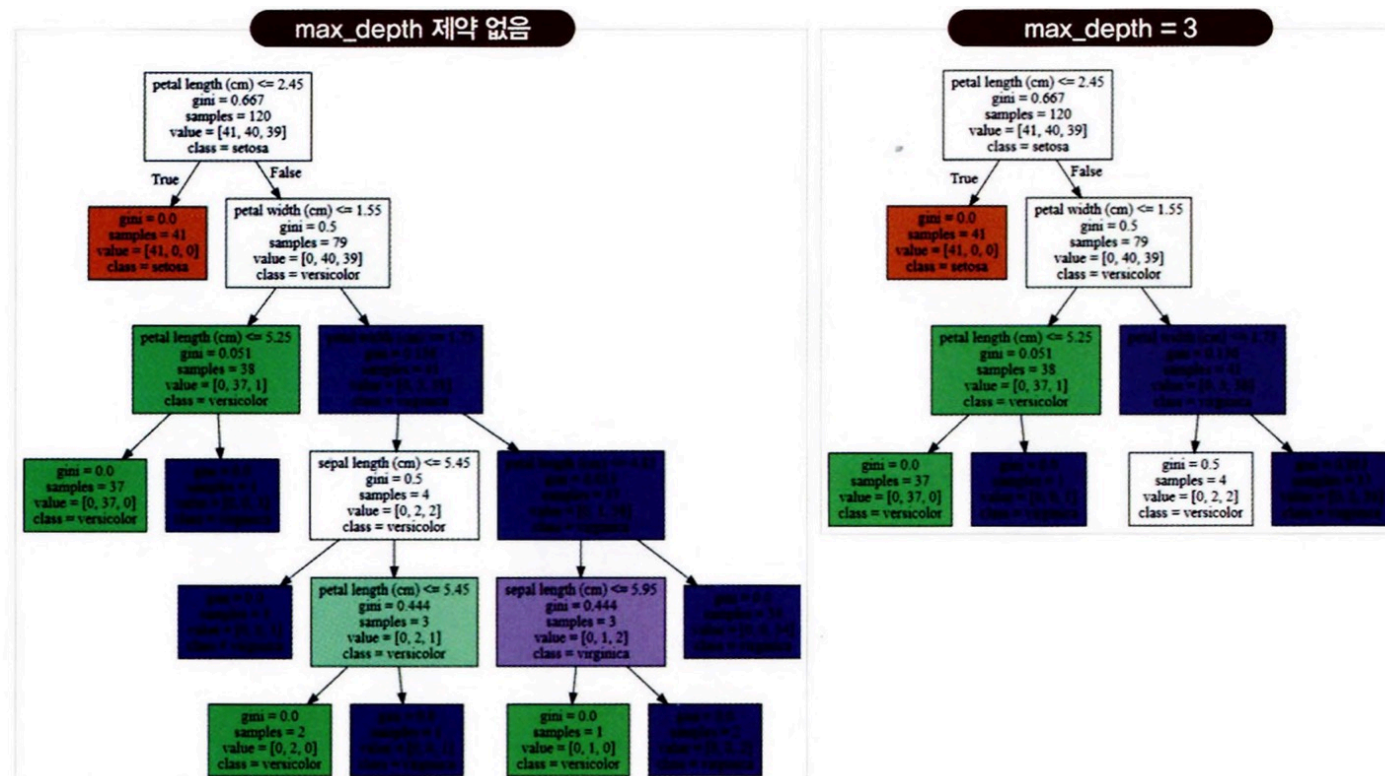


1. `petal length (cm) <=2.45` 규칙이 분기하게 되면 2번, 3번 노드가 만들어짐

2. 2번 노드는 모든 데이터가 Setosa → 클래스로 결정된 리프 노드가 됨 (지니계수가 0)
3. 3번 노드는 `petal length (cm) <= 2.45` 가 False인 규칙 노드
 - a. 79개의 샘플, 지니계수가 0.5로 높으므로 다음 분기를 위한 규칙 필요
 - b. `petal width (cm) ≤ 1.55` 규칙으로 자식 노드 생성
4. 4번 노드는 `petal width (cm) <= 1.55` 가 True인 규칙 노드
 - a. 지니 계수는 0.051로 매우 낮으나 여전히 혼재되어 있으므로 새로운 규칙으로 다시 자식 노드 생성
5. 5번 노드는 `petal width (cm) <= 1.55` 가 False인 규칙 노드
 - a. 지니 계수는 0.136으로 낮으나 여전히 혼재되어 있으므로 새로운 규칙으로 다시 자식 노드 생성
- 각 노드의 색깔은 붓꽃 데이터의 레이블 값을 의미
- 주황색은 Setosa, 초록색은 Versicolor, 보라색은 Virginica
 - 색이 짙어질수록 지니 계수가 낮고 해당 레이블의 샘플데이터가 많음

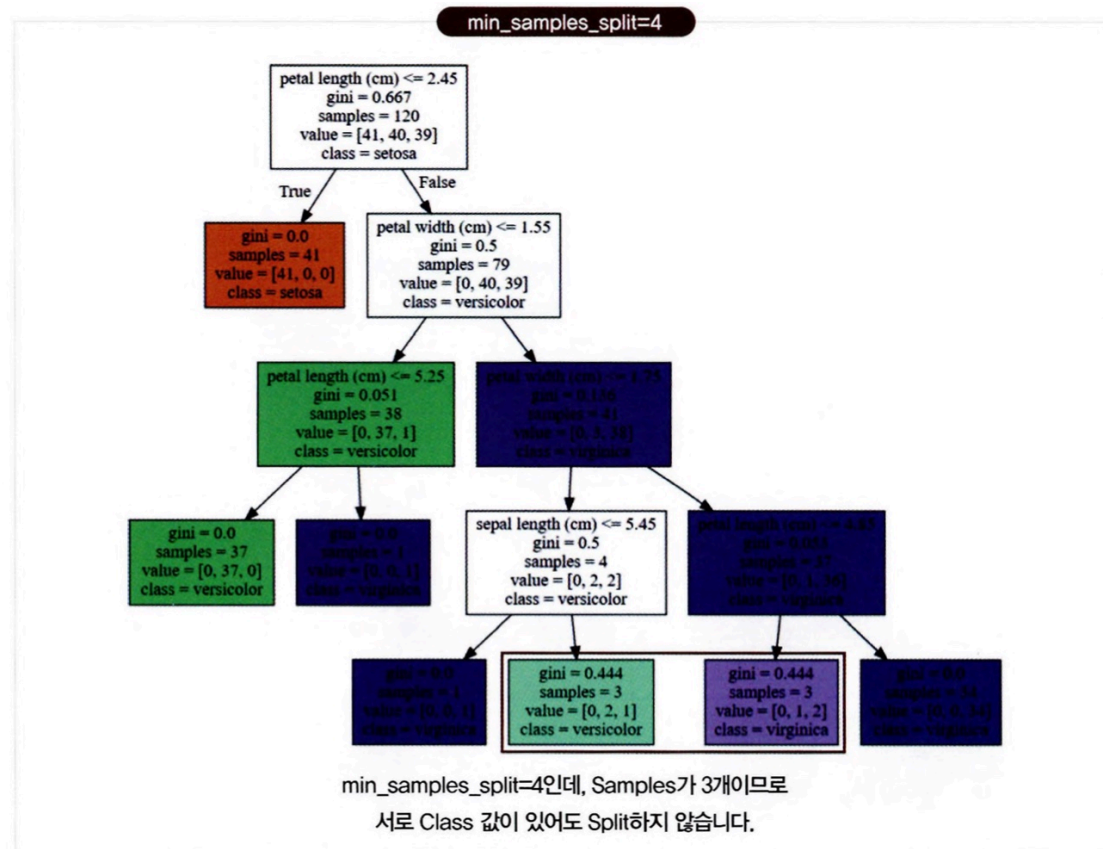
하이퍼 파라미터 변경에 따른 트리 변화

- `max_depth`



⇒ `max_depth` 를 제한 없음에서 3개로 설정하면 깊이가 줄어들면서 더 간단한 결정 트리가 됨

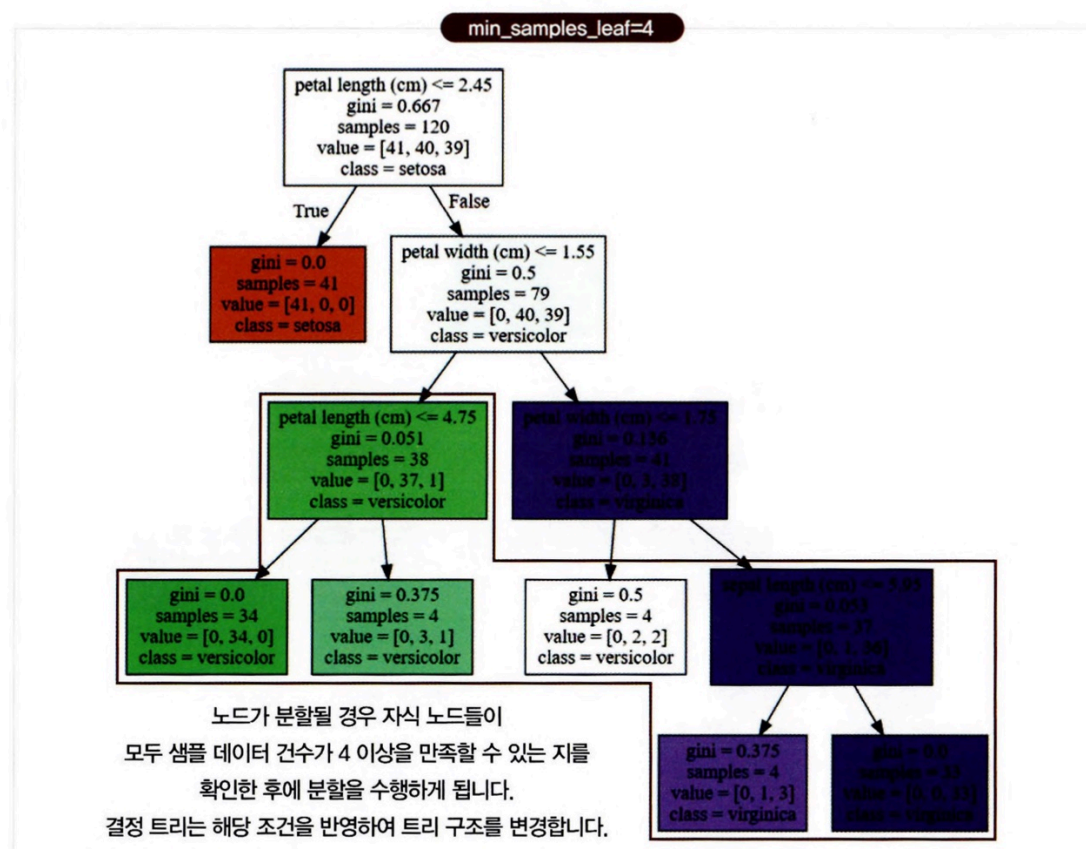
- `min_samples_split`



min_samples_split = 4 인 경우

⇒ 자식 노드로 분할하기 위해서는 최소 4개의 샘플이 필요한데, 없으면 리프 노드가 됨

- **min_samples_leaf**



min_samples_leaf = 4 인 경우

⇒ 어떤 노드가 분할할 경우, 왼/오른쪽 자식 노드 모두 4개 이상의 데이터를 가져야 함

⇒ 분할 조건이 어려워지므로, 상대적으로 적은 횟수로 분기

피쳐의 중요도

결정 트리는 균일도에 기반, 어떤 속성을 규칙 조건으로 선택하느냐가 중요!

⇒ 사이킷런의

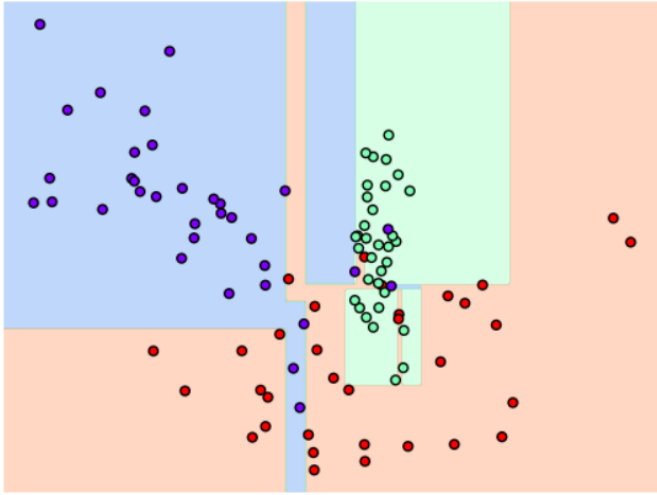
`feature_importances_` 속성으로 확인 가능

```
# feature importance 추출
print("Feature importances:\n{0}".format(np.round(dt_clf.feature_importances_, 3)))
```

- ndarray 형태로 값을 반환 (피쳐 순서대로 값 할당)
- 피쳐가 트리 분할 시, 정보 이득/지니 계수를 얼마나 효율적으로 잘 개선 시켰는지
- 일반적으로 값이 높을수록 중요도가 높다는 의미

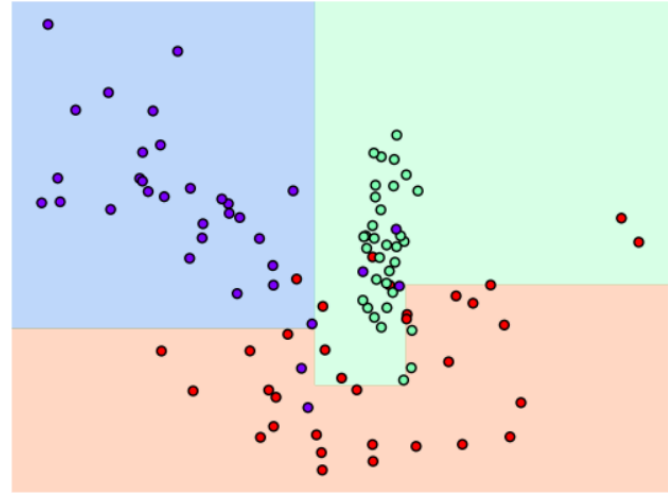
결정 트리 과적합

하이퍼파라미터값이 디폴트인 Classifier의 결정 기준 경계



일부 이상치(Outlier)데이터까지 분류하기 위해 분할이 자주 일어나서 결정 기준 경계가 매우 많아짐

min_sample_leaf = 6으로 설정한 Classifier의 결정 기준 경계



더 일반화된 분류 규칙에 따라 분류가 됨

⇒ 트리 생성 조건을 제약한 모델이 새로운 데이터에 대한 정확도가 더 뛰어날 가능성이 높다 !

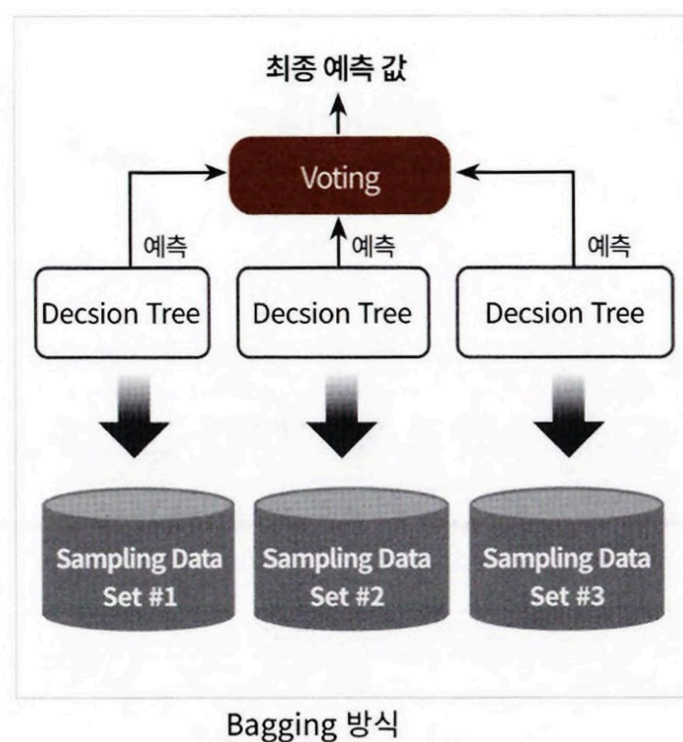
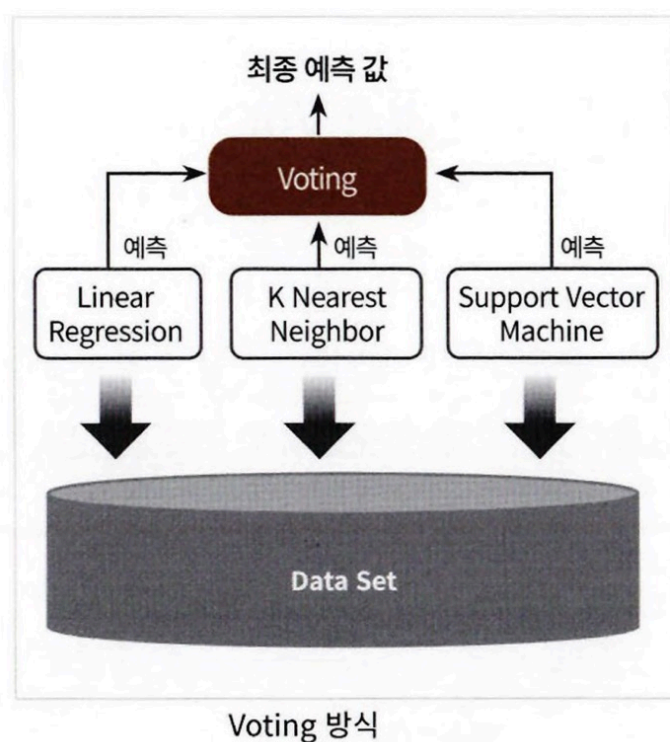
3 앙상블 학습

✓ 앙상블 학습(Ensemble Learning)

여러 개의 분류기(Classifier)를 생성 → 예측들을 결합함으로써 보다 정확한 최종 예측을 도출해내는 기법 (= 집단 지성)

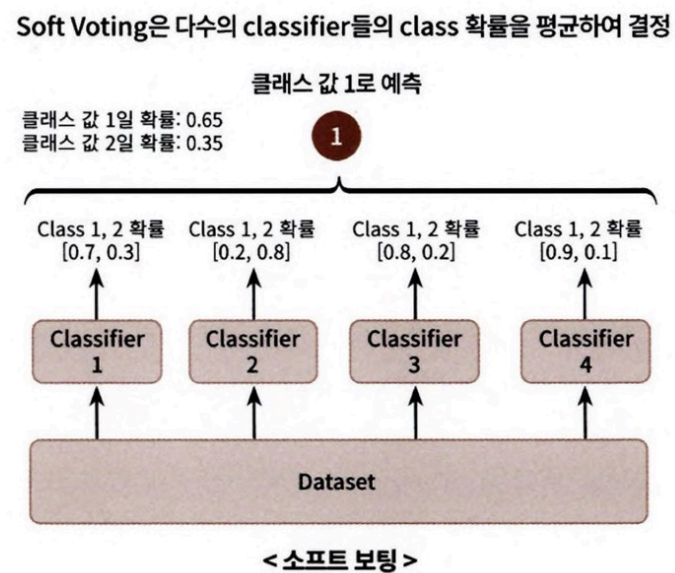
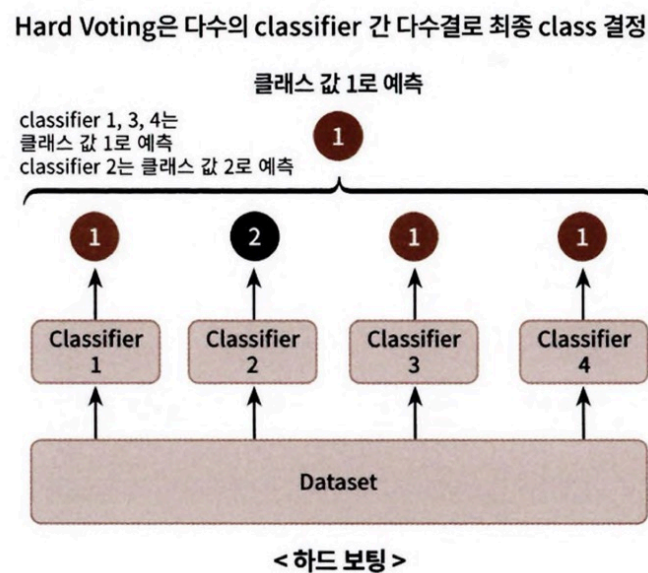
- 비정형 데이터의 분류에는 딥러닝이 뛰어난 성능을 보이고 있지만, 대부분의 정형 데이터 분류의 경우 앙상블이 뛰어남
- 쉽고 편하면서도 강력한 성능을 보유

앙상블 방식



- **보팅 (Voting)**
 - 서로 다른 알고리즘을 가진 분류기를 결합
- **배깅 (Bagging)**
 - 같은 유형의 알고리즘을 기반으로한 분류기 + 서로 다른 데이터 샘플링으로 보팅을 수행 (e.g. 랜덤 포레스트)
 - 부트스트래핑(Bootstrapping)
원본 학습데이터를 샘플링해 추출 → 개별 Classifier에 할당
 - 교차 검증과 달리, 데이터 세트 간 중첩 허용
- **부스팅 (Boosting)**
 - 여러 개의 분류기가 순차적으로 학습
→ 먼저 학습한 분류기가 틀린 데이터에 대해 가중치(weight)를 부여하며 학습과 예측을 진행
 - 예측 성능이 뛰어나 앙상블 학습을 주도하고 있는 방법 (e.g. 그래디언트 부스트, XGBoost, LightGBM)
- **스태킹 (Stacking)**
 - 여러가지 다른 모델의 예측 결과값
→ 다시 학습데이터로 만들어 다른모델(메타모델)로 재학습/예측

보팅 유형 - Hard Voting과 Soft Voting



- 다수결 원칙과 비슷
→ 다수가 결정한 예측값을 최종 선택
- 확률을 평균내서 가장 높은 레이블을 선택

⇒ 일반적으로 소프트 보팅이 예측 성능이 더 좋아서 많이 사용됨

보팅 분류기 (Voting Classifier)

사이킷런은 보팅 방식의 앙상블을 구현한 `VotingClassifier` 클래스를 제공하고 있음

```
from sklearn.ensemble import VotingClassifier
```

- 주요 인자
 - estimator : 보팅에 사용될 여러 classifier 객체들의 리스트값
 - voting : 'hard'시 하드보팅, 'soft'로 설정 시 소프트 보팅 방식 적용 (기본은 hard)
- 단일 알고리즘보다 뛰어난 예측 성능을 보이는 경우가 많음
 - 다양한 알고리즘들이 서로 결합해 유연성을 가지고 현실 데이터에 대처할 수 있기 때문
- 결정 트리 알고리즘 기반

- 결정 트리 알고리즘의 단점 (과적합 발생 가능성) 을 많은 분류기를 결합해 극복
- 결정 트리 알고리즘의 장점은 취하고 단점을 보완하며 편향-분산 트레이드오프의 효과를 극대화

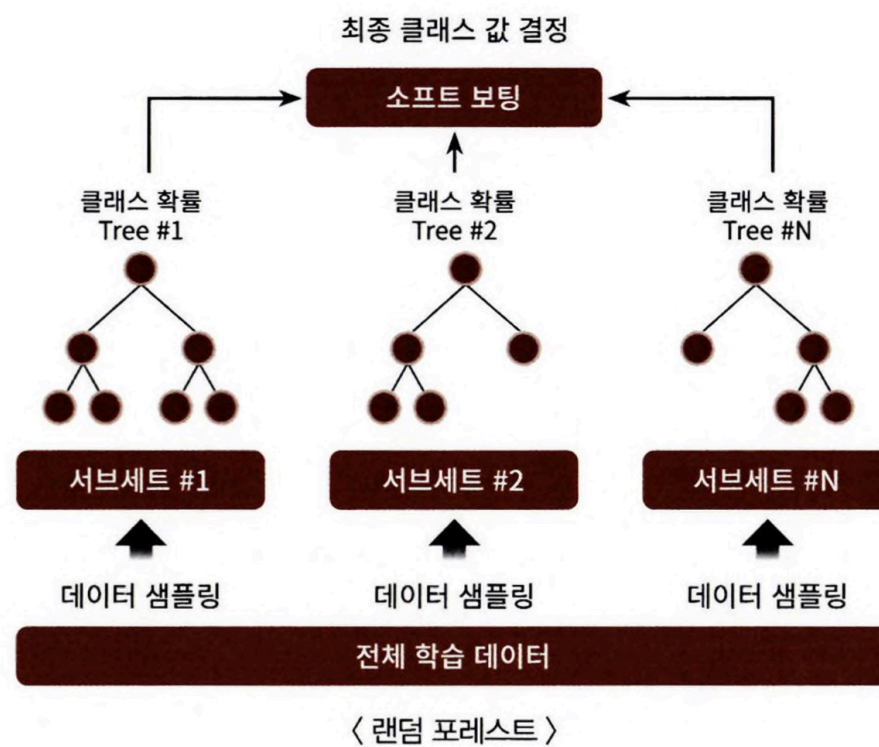
4 랜덤 포레스트

개요 및 실습



랜덤 포레스트

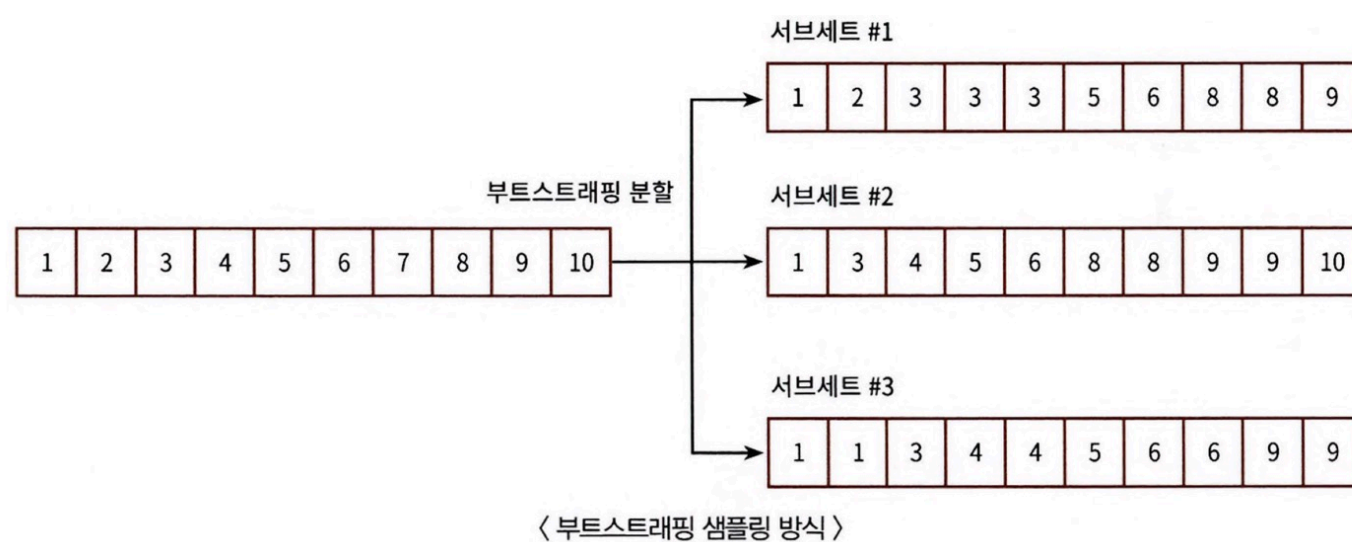
같은 알고리즘으로 여러 개의 분류기를 만들어서 보팅으로 결정하는 **배깅**의 대표적인 알고리즘



- 여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링
 - 학습 수행 후 모든 분류기가 보팅을 통해 예측 결정

부트스트래핑(bootstrapping) 분할 방식

전체 데이터 세트에서 일부가 중첩되도록 샘플링해서 개별 트리가 학습하는 데 사용



하이퍼 파라미터 `n_estimators = 3`으로 부여했을 때

- ⇒ 서브 세트의 데이터 건수는 전체 데이터와 동일하지만, 개별 데이터가 중첩되어 만들어짐
- ⇒ 사이킷런에서는 `RandomForestClassifier` 클래스에서 지원

하이퍼 파라미터 및 튜닝

- 트리 기반의 앙상블 알고리즘의 단점
→ 하이퍼 파라미터가 너무 많고, 튜닝을 위한 시간이 많이 소요됨
- 그나마 랜덤 포레스트가 적은 편 (결정 트리에서 사용되는 파라미터와 대부분 같다)

하이퍼 파라미터	설명
<code>n_estimators</code>	랜덤 포레스트에서 결정 트리의 개수 지정 (디폴트는 10) → 계속 증가시킨다고 성능이 무조건 향상 X / 수행 시간 증가
<code>max_features</code>	결정 트리에서의 <code>max_features</code> 파라미터와 같음 하지만 디폴트는 'auto' → 피쳐 참조시 <code>sqrt(#)</code> 만큼 참조함 (전체 피쳐가 16개라면 분할을 위해 4개 참조)
<code>max_depth</code> , <code>min_samples_leaf</code> , ...	결정 트리에서 과적합 개선을 위해 사용되는 파라미터를 똑같이 적용 가능

- 결정 트리와 같이 `feature_importances_` 속성을 이용해 피쳐 중요도 확인 가능